ResQFlow-Flood

Sensing & Continuous Feedback - Suggestions and Priorities

Information storage base for the revised urban flood evacuation project.
Purpose: decision-support demo with software-only sensing (no IoT hardware budget).
Status: planning notes - implement in priority order after team agreement.

1. Project framing (keep this honest)

ResQFlow-Flood is closed-loop decision support for urban flood evacuation: move at-risk groups to capacity-limited shelters using buses, high-clearance vehicles, and boats. Feedback is generated in software (simulator + human report forms + optional public weather APIs). This is not a live city flood-control system and not autonomous emergency command.

Strong mentor line: Public weather gives regional context; citizens give early street reports; operators give high-trust fast feedback. Actuation and 8-check verification stay in the closed-loop dispatcher. Hardware sensors would later plug into the same report API.

2. The two bottlenecks

Bottleneck A - Initial sensing (input)

Problem: Where does the system get the disaster? Mentors need a believable Sense step before dispatch runs.

Need: Something enters the plant - citizen report, weather context, or operator report - so the map and priorities are not only hard-coded scenario JSON.

Bottleneck B - Continuous / fast feedback

Problem: The loop must stay closed as the flood changes - plans invalidate mid-mission.

Need: Frequent updates that can invalidate or repair a plan. 'Instantaneous' for this project means: operator submits in 1-2 seconds -> state updates -> re-rank / repair on the next decision cycle (or immediately on high-priority reports). It does NOT require satellite GPS or Google Maps as the core mechanism.

3. Initial sensing options

3.1 Citizen report form (already started on flood)

- Feasible: YES - best first move. Free, live-demoable, maps to real crowdsourced disaster apps.
- Use for: street waterlogging started; people need help at a location.
- Limit: noisy (duplicates, wrong location). Can later become a feature (stale/duplicate handling).
- Verdict: Keep and polish. Primary Sense story for review.
- Current artifact: report.html + POST /flood/report (Part A).

3.2 Public weather / flood APIs

- Feasible: YES for context - NOT for street-level rescue claims.
- Examples: Open-Meteo, OpenWeather (rain/alerts); optional gov river gauges if published.
- Good for: why flooding is likely; nudge simulator rainfall; dashboard banner.
- Bad for: exact street depth; which underpass is closed.
- Verdict: Second Sense channel. Do not claim weather API replaces street reports.

4. Continuous feedback options

4.1 Operator high-priority form (strongly recommended next)

- Feasible: YES - demo gold.
- Citizen = lower/medium trust; Operator/field unit = high trust, higher priority for reinforcements.
- One-tap actions: road closed; need reinforcements; shelter full; vehicle cannot proceed; urgency 1-5.
- Maps to real incident command: official reports outrank public tips.
- Verdict: Highest value next build after citizen form.

4.2 Click-on-map feedback (ops dashboard)

- Feasible: YES. Faster than a separate form.
- Operator clicks cell/road -> mark flooded / request unit.
- Verdict: Nice follow-on after operator form.

4.3 Browser GPS / phone geolocation

- Feasible as a toy; WEAK as a core claim.
- Problems: map is a simulated grid, not real lat/lng; classroom Wi-Fi/permissions flaky.
- Verdict: Optional later only - not the feedback backbone.

4.4 Google Maps integration

- Technically possible; mostly NOT worth it for next review.
- Costs: API keys, billing, time; still no water depth; pulls focus from CPS loop to map product.
- Verdict: Skip unless faculty explicitly asks for GIS.

4.5 Photos of flooding

- Feasible as evidence attachment + manual severity by operator.
- NOT feasible (without money/time): reliable CV that measures depth from a phone photo.
- Honest framing: photo supports an operator report; severity is selected by the human.
- Verdict: Optional later. Do not make vision the feedback path for the next review.

5. Priority ladder (do in this order)

P0 (now). Citizen form on flood demo

Do it? YES - already started

Why: Solves initial Sense; live mentor demo.

P1 (next). Operator high-priority form + trust / priority weighting

Do it? YES - build next

Why: Solves fast continuous feedback; high-trust reinforcements outrank citizen noise.

P2 (if one evening free). Public weather API banner / rainfall nudge

Do it? YES - low cost

Why: Shows public-data sensing without claiming street accuracy.

P3 (follow-on). Click-map operator actions + report log (citizen vs operator)

Do it? Nice to have

Why: Makes feedback feel more instantaneous on the ops UI.

P4 (immediate replan). High-priority operator report triggers replan without waiting many ticks

Do it? Do after P1

Why: Strengthens closed-loop story.

Defer / skip for next review. Google Maps, GPS-as-core, photo -> automatic depth AI, real IoT sensors

Do it? NO for now

Why: Cost, scope creep, and honesty risk with mentors.

6. Feasibility summary

Citizen form - Feasible now - Primary initial Sense.

Operator priority form - Feasible next - Primary fast feedback.

Weather API (context only) - Feasible soon - Secondary Sense.

Map click feedback - Feasible after P1 - UX speed.

Browser GPS - Weak core - Optional toy later.

Google Maps - Skip now - Time/billing/focus cost.

Photo AI depth - Not for review - Optional evidence attachment only.

Real IoT sensors - Out of budget - Same API later if funded.

7. What to avoid claiming in review

- Public weather equals street-level rescue sensing.
- Photos measure water depth without a real validated model.
- Building Google Maps before the operator priority lane.
- Mixing simulated grid plant with real city streets without saying which is which.
- Implying autonomous emergency command or live municipal deployment.

8. Suggested mentor narrative (30 seconds)

We have three Sense channels: public weather for regional context, citizens for early street reports, and operators for high-trust fast feedback. The dispatcher still prioritizes with Flood-GAPD, ranks vehicle-route-shelter options, verifies eight flood checks, then actuates. If roads close or shelters fill, feedback invalidates the plan and we repair or fail safely. Hardware sensors are out of budget; they would use the same

report API later.

9. Relation to general demo vs flood demo

Same CPS pattern, different nouns. Citizen/operator/weather Sense can feed flood (waterlogging, groups, shelters) or the general demo (incidents/resources). Prefer prototyping Sense+feedback on the flood specialization first; general demo index.html remains on GitHub (origin/main) as a safe rollback.

GitHub fallback: https://github.com/abhinav429/ResQFlow - restore general demo with: git checkout origin/main -- index.html

10. Document control

Created as an internal information base for the revised ResQFlow-Flood project.

Complements ARCHITECTURE-FLOOD.md and the implementation plan with Sense/Feedback priorities.

Next build recommendation when the team is ready: P1 Operator high-priority form.